# COSC 423/523: Introduction to Artificial Intelligence

# Markov Decision Processes

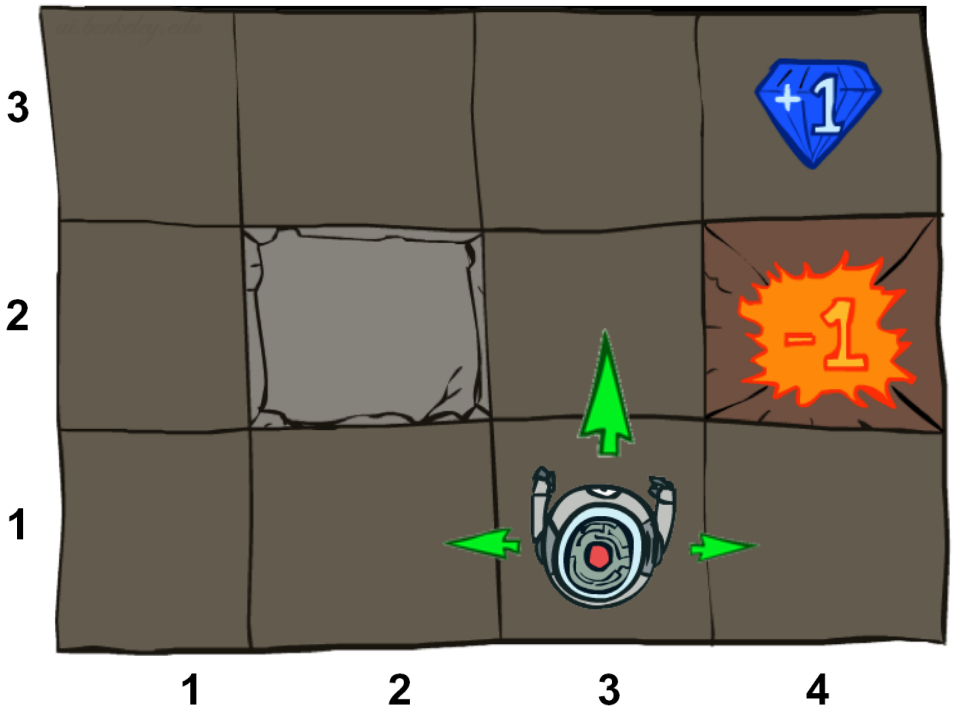


THE UNIVERSITY OF TENNESSEE KNOXVILLE

Credits to UC Berkeley CS188 (Introduction to Artificial Intelligence), with thanks to Professor Pieter Abbeel for sharing the materials.

# Recap: Grid World

- A maze-like problem
  - The agent lives in a grid
  - Walls block the agent's path
- Noisy movement: actions do not always go as planned
  - 80% of the time, the action North takes the agent North (if there is no wall there)
  - 10% of the time, North takes the agent West; 10% East
  - If there is a wall in the direction the agent would have been taken, the agent stays put
- The agent receives rewards each time step
  - Small "living" reward each step (can be negative)
  - Big rewards come at the end (good or bad)
- Goal: maximize sum of rewards



# Recap: Solving MDP

- The value (utility) of a state s:

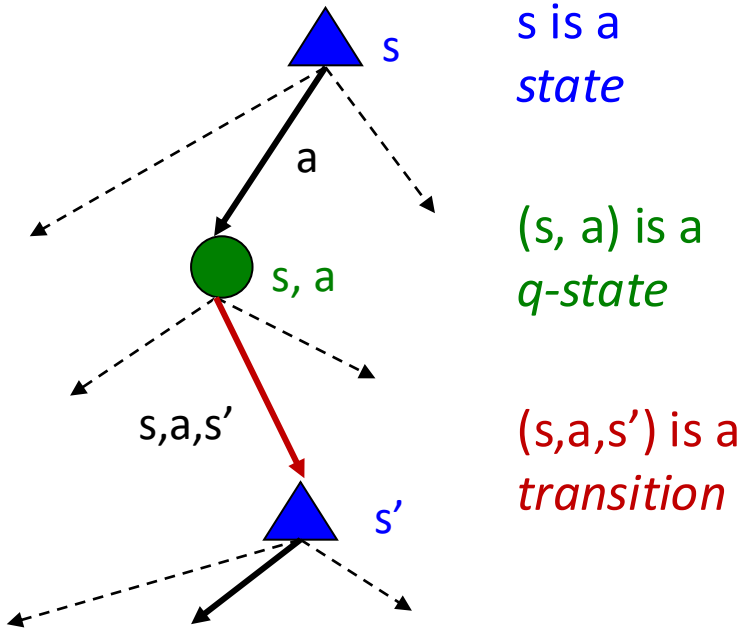
  $V^*(s)$ = expected utility starting in s and acting optimally

- The value (utility) of a q-state (s,a):

  $Q^*(s,a)$ = expected utility starting out having taken action a from state s and (thereafter) acting optimally

- The optimal policy:

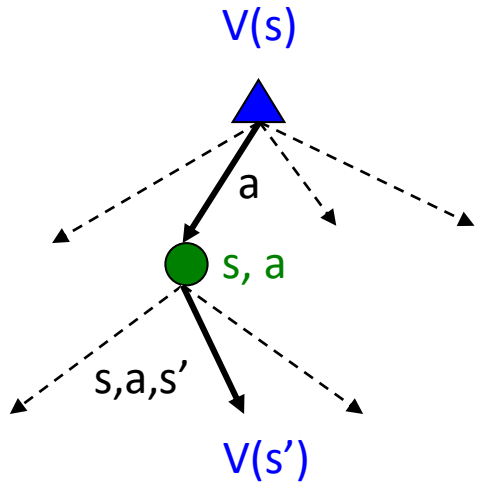
  $\pi^*(s)$ = optimal action from state s



# Recap: Value Iteration

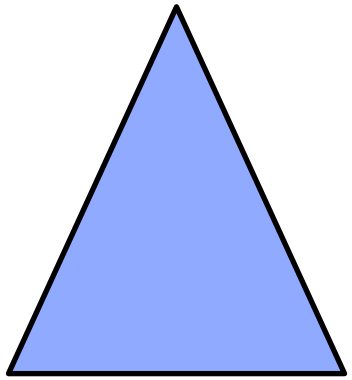
- Bellman equations characterize the optimal values:

$$V^*(s) = \max_a \sum_{s'} T(s,a,s') \left[ R(s,a,s') + \gamma V^*(s') \right]$$




- Value iteration computes them:

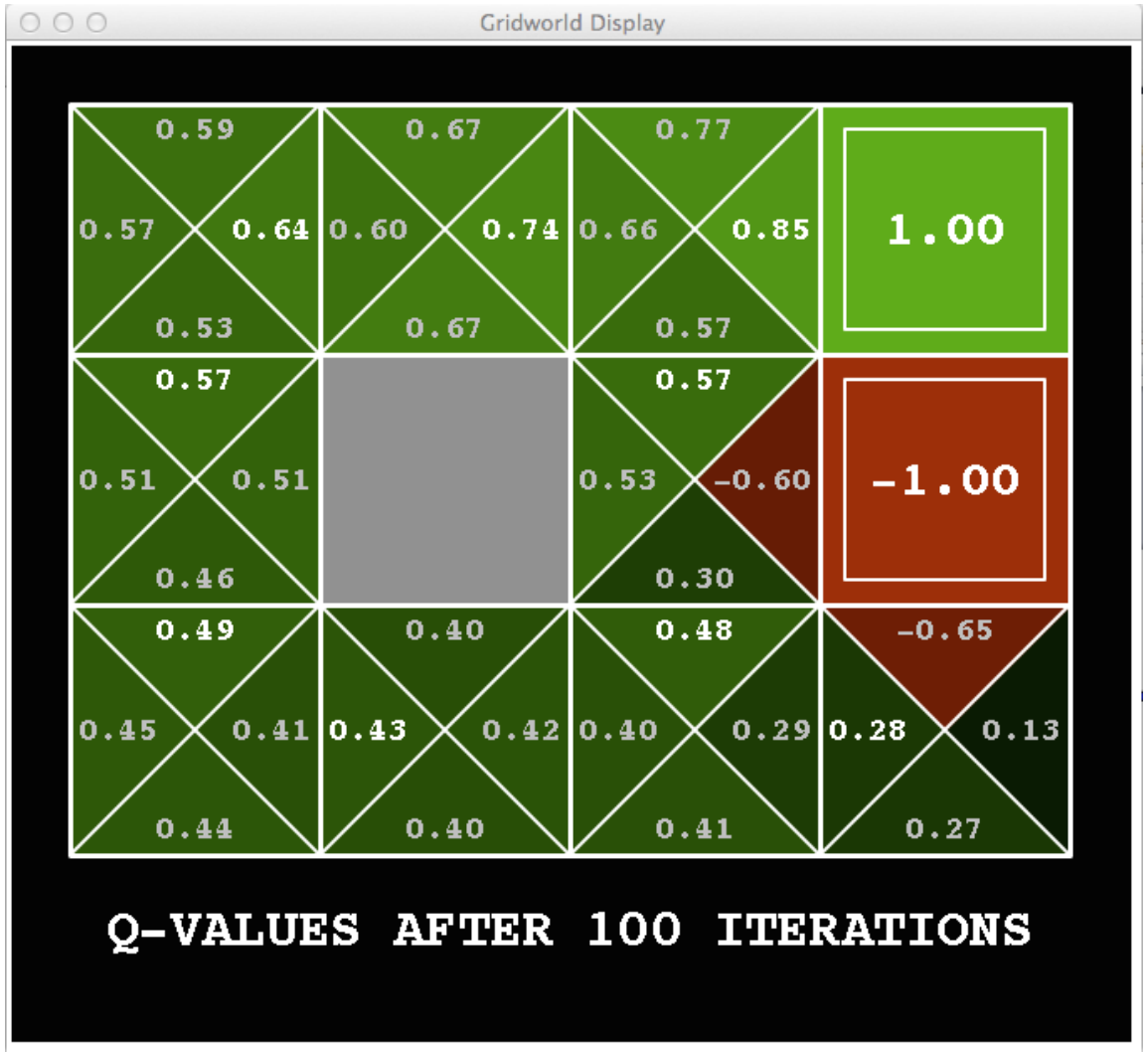
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[ R(s,a,s') + \gamma V_k(s') \right]$$



- Value iteration is just a fixed point solution method
  - $V_k$ can also be seen as the values of depth-$k$ truncated trees
  - **Theorem.** If $0 \leq \gamma < 1$, value iteration converges to the unique optimal value function $V^*$

# Gridworld V Values

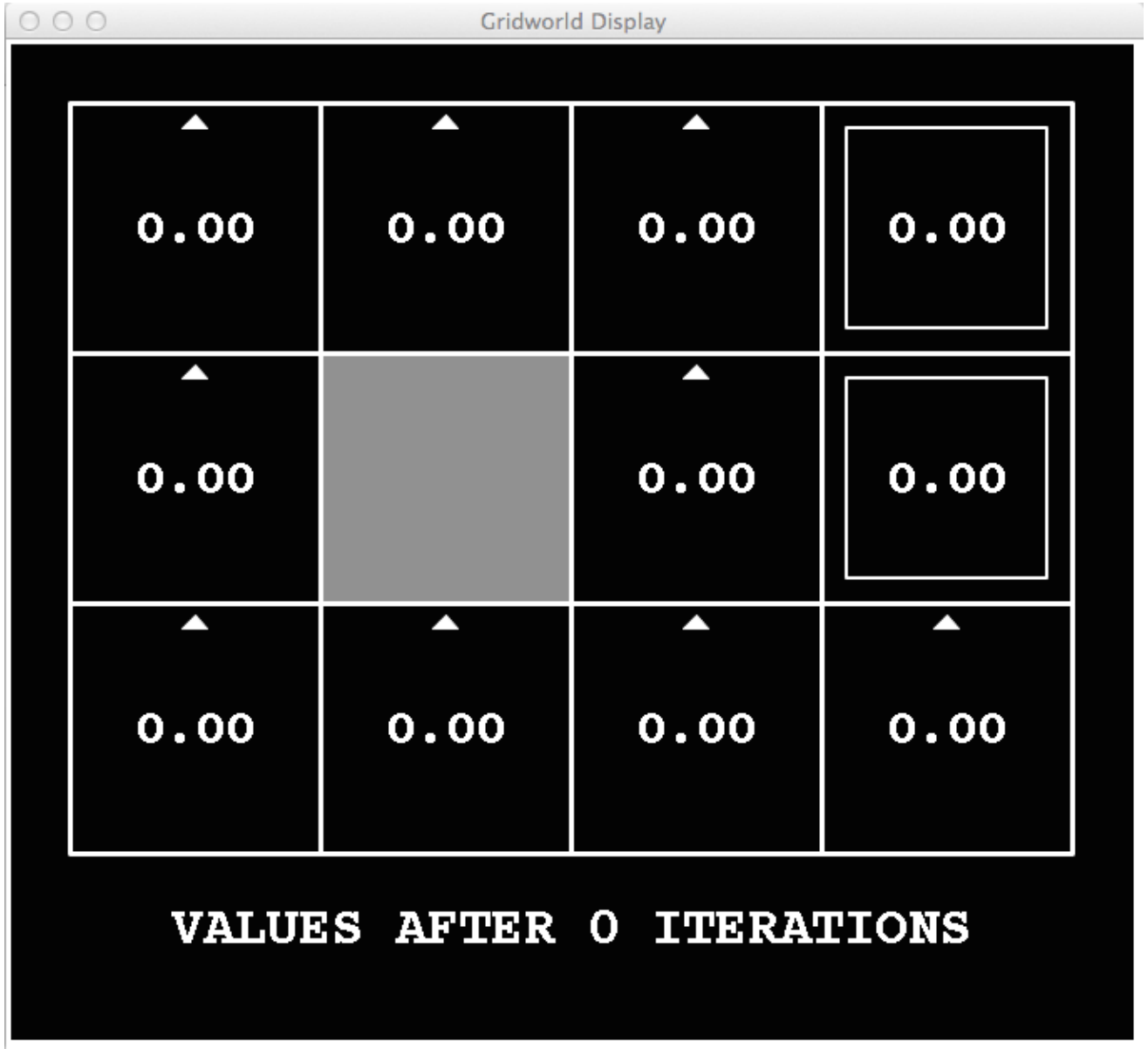



Demo: https://nowke.github.io/rlviz

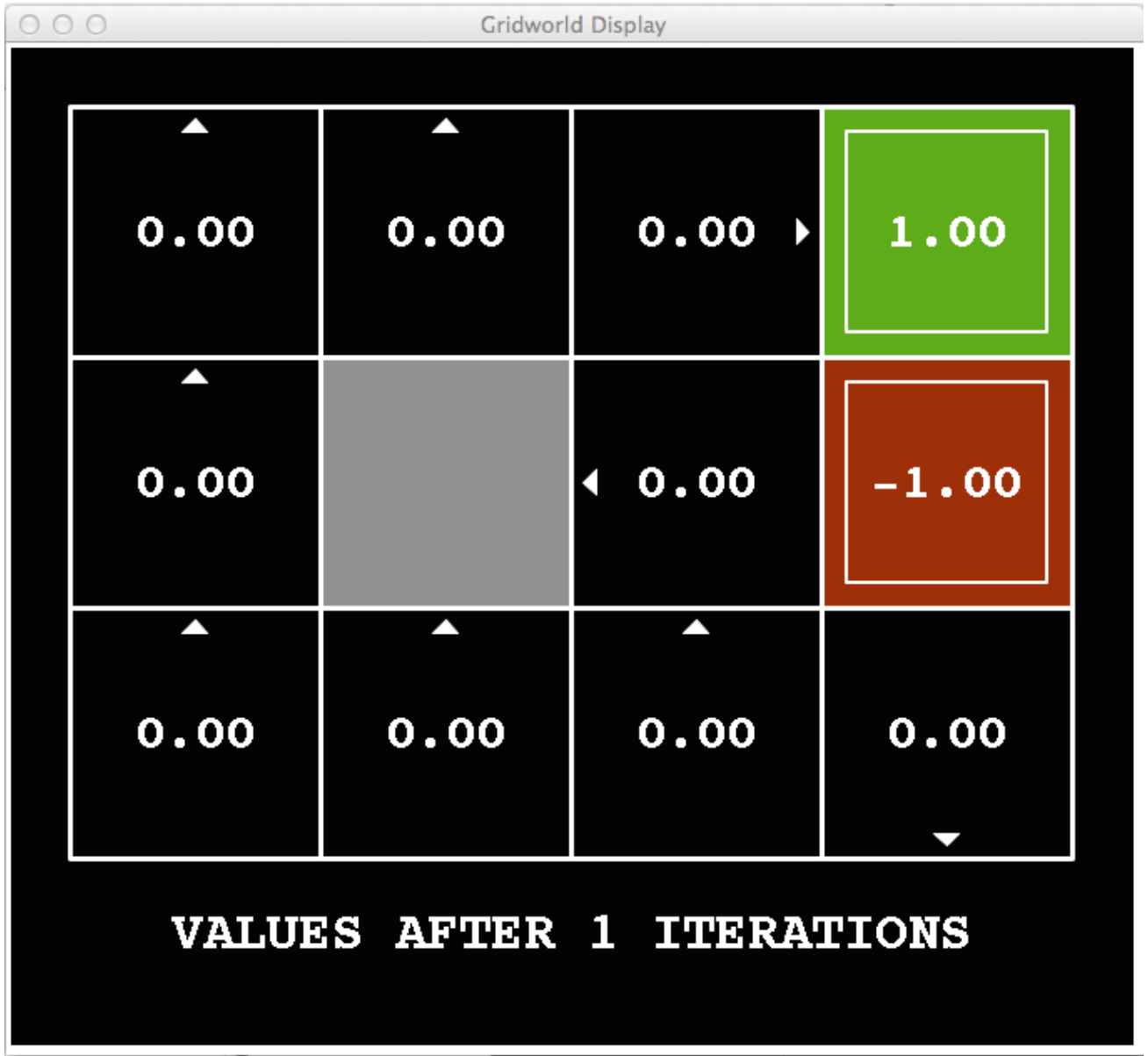
Noise = 0.2, Discount = 0.9, Living reward = 0

# k=0




Noise = 0.2
Discount = 0.9
Living reward = 0

# k=1



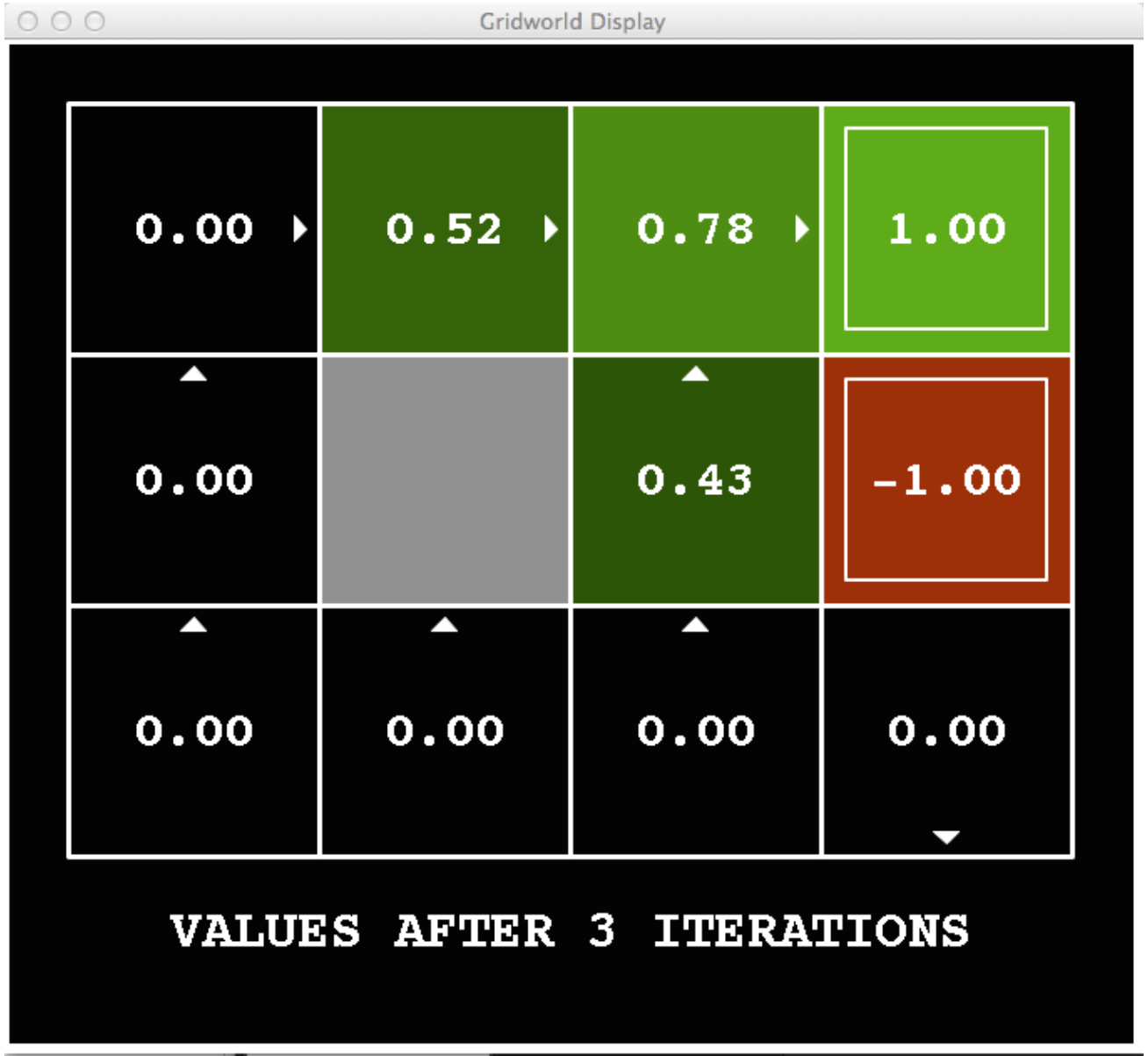

Noise = 0.2
Discount = 0.9
Living reward = 0

# k=2



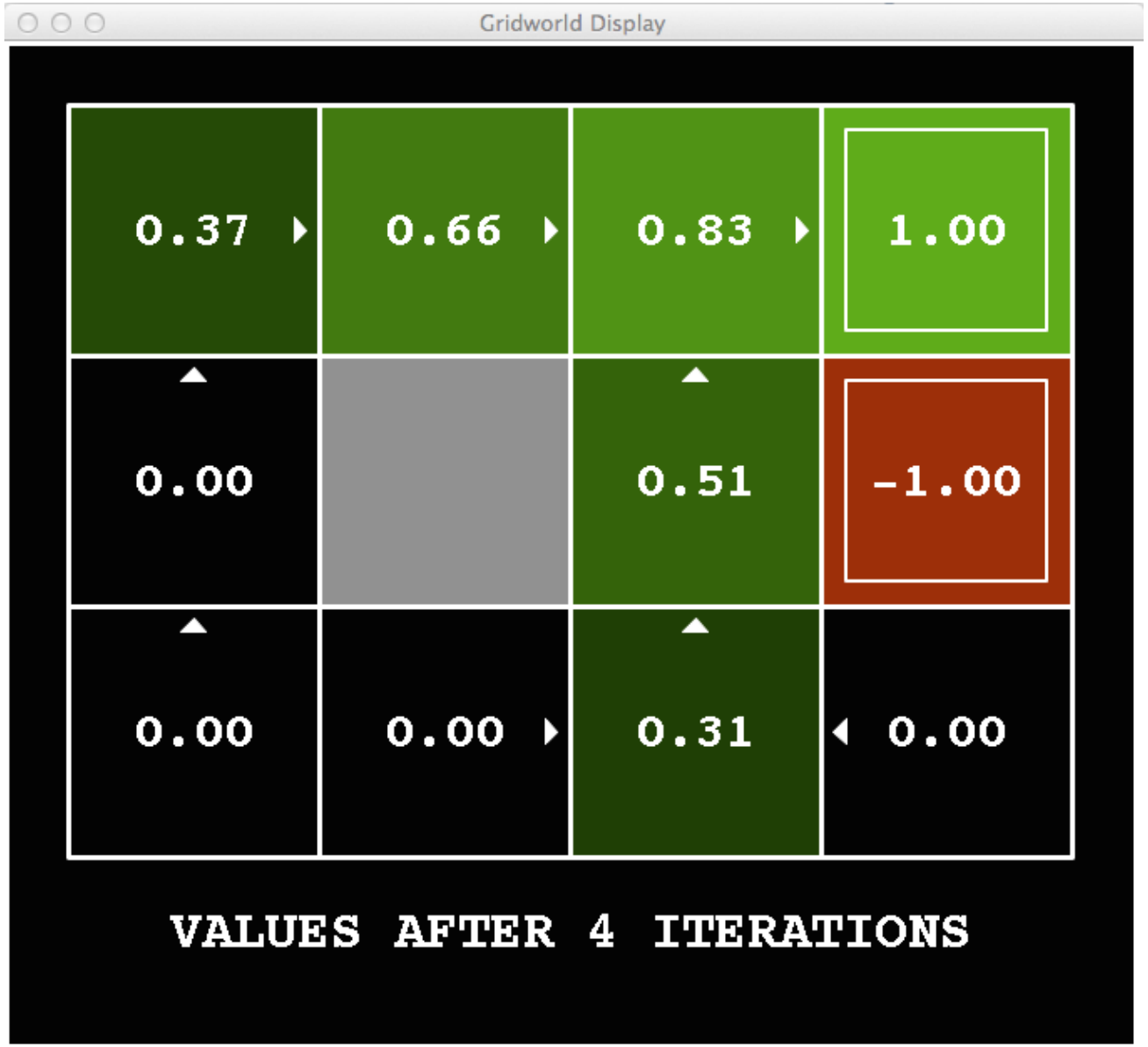

Noise = 0.2
Discount = 0.9
Living reward = 0

# k=3



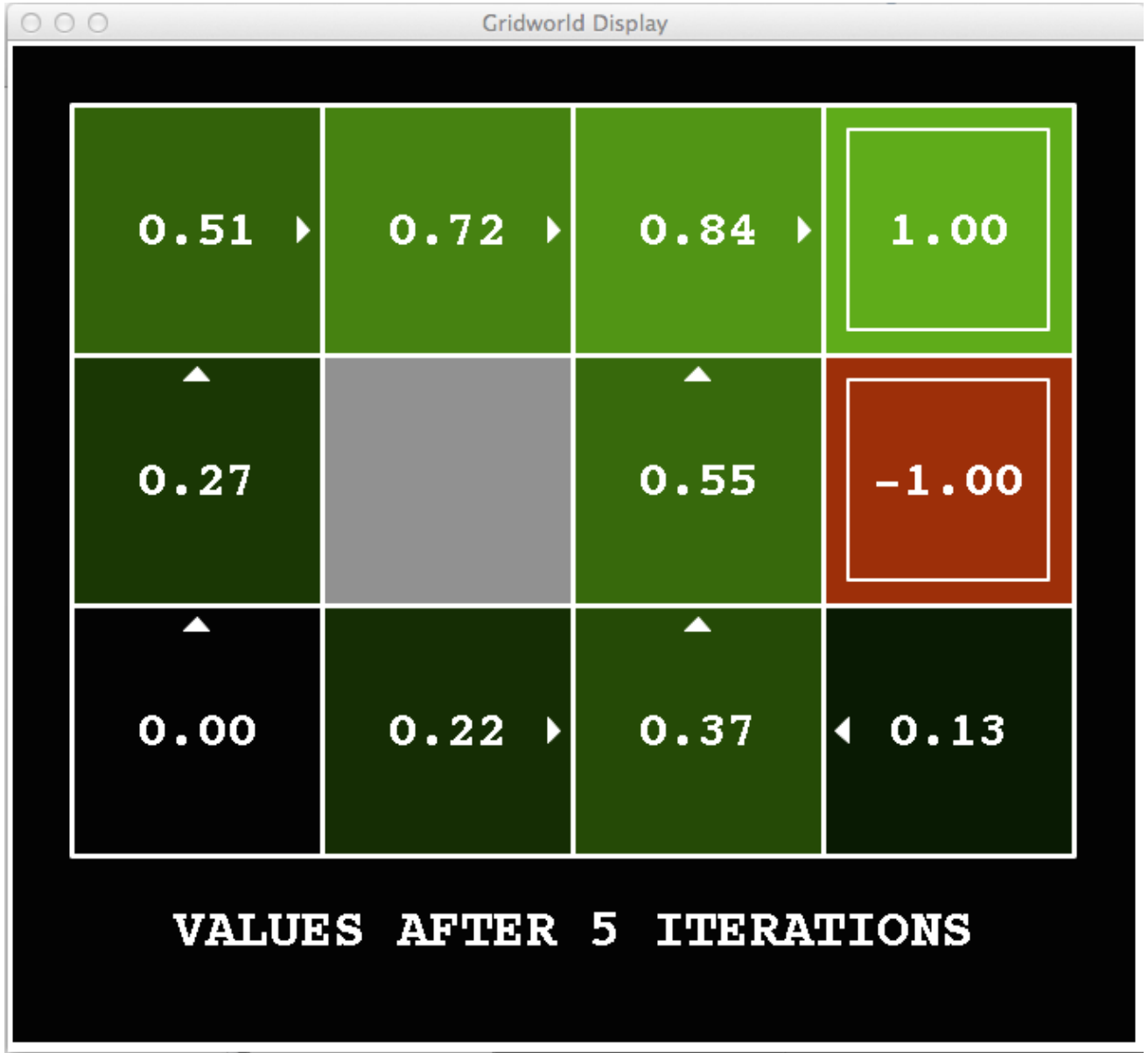

Noise = 0.2
Discount = 0.9
Living reward = 0

# k=4



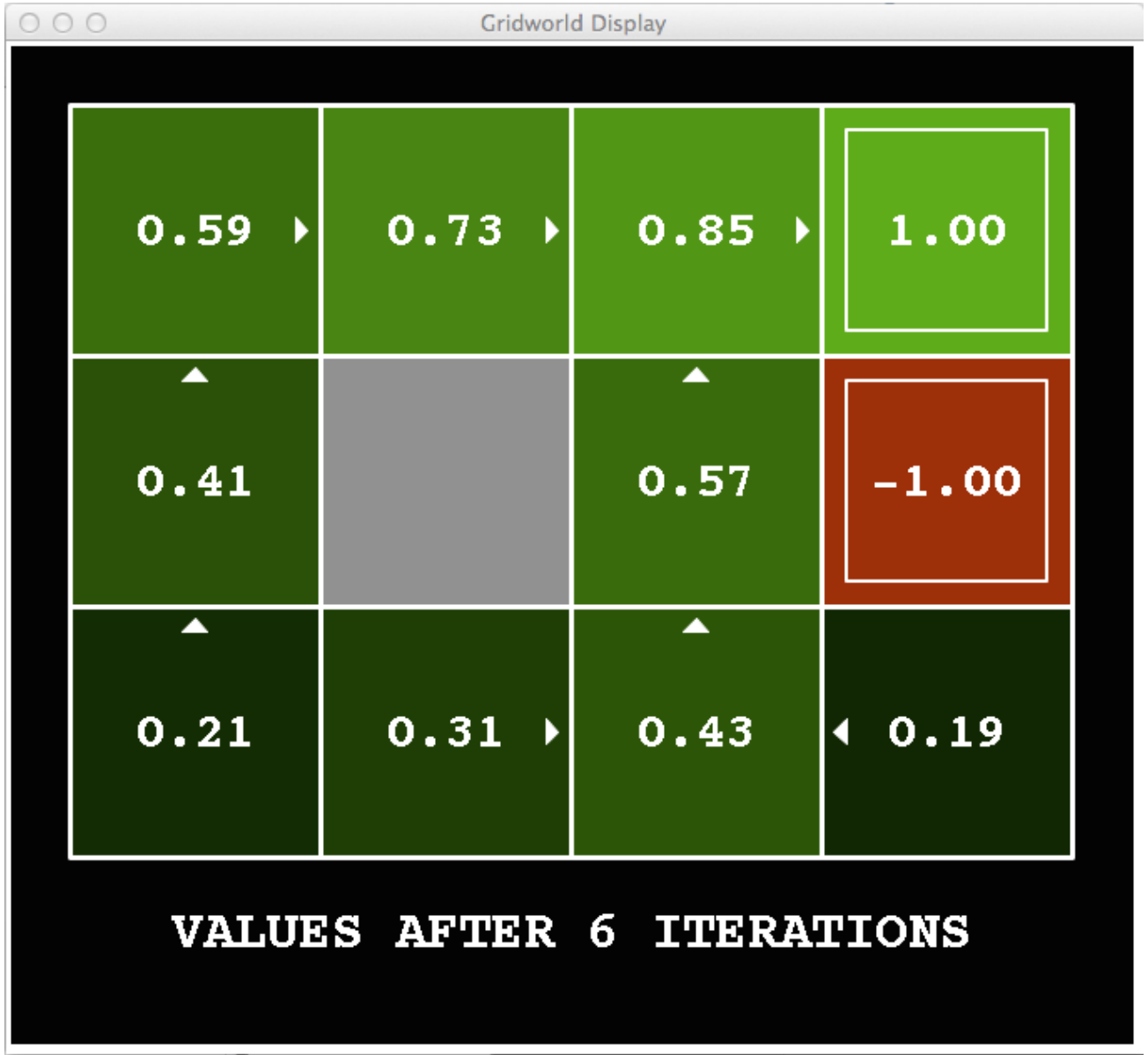

Noise = 0.2
Discount = 0.9
Living reward = 0

# k=5



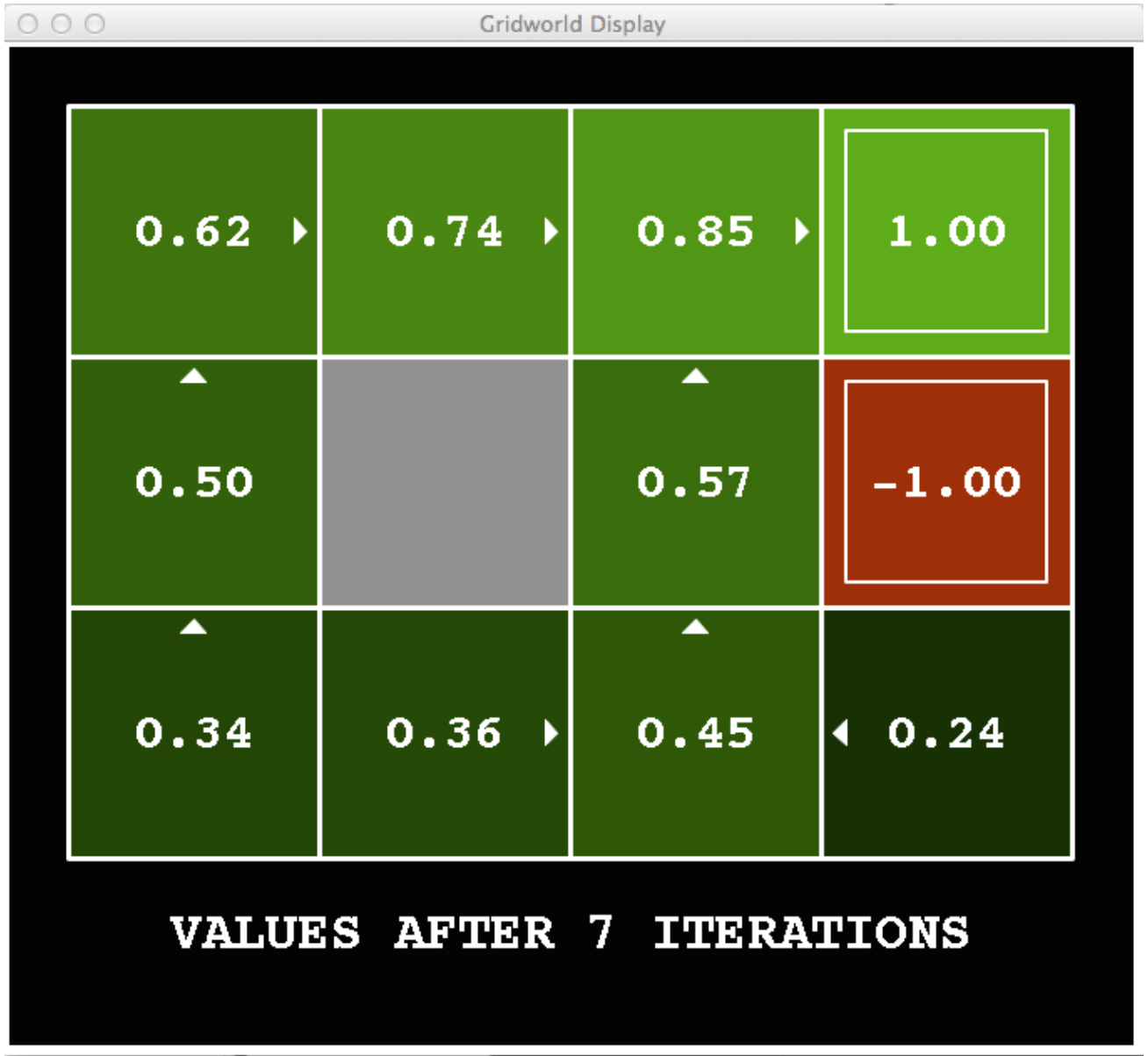

Noise = 0.2
Discount = 0.9
Living reward = 0

# k=6



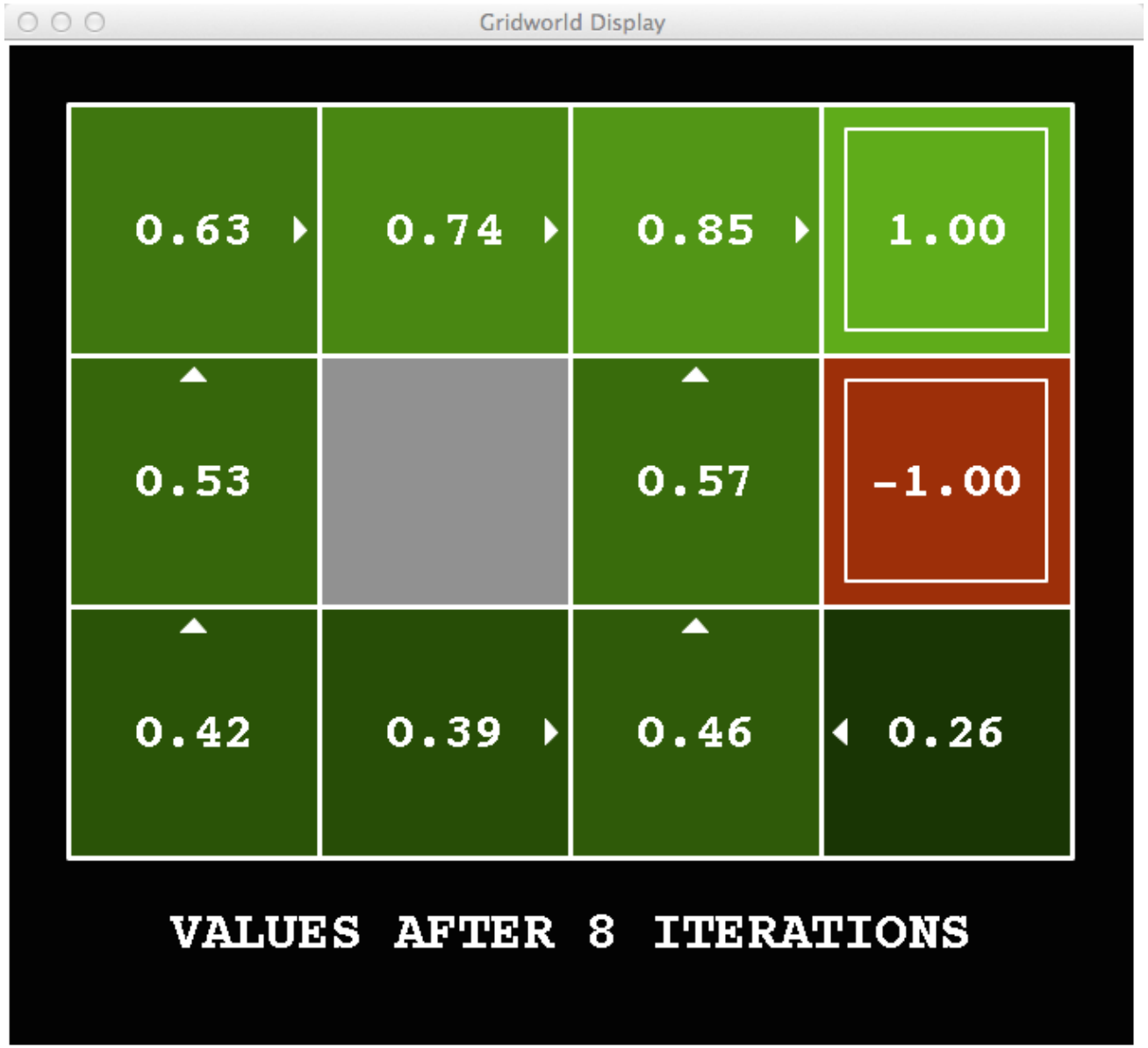

Noise = 0.2
Discount = 0.9
Living reward = 0

# k=7



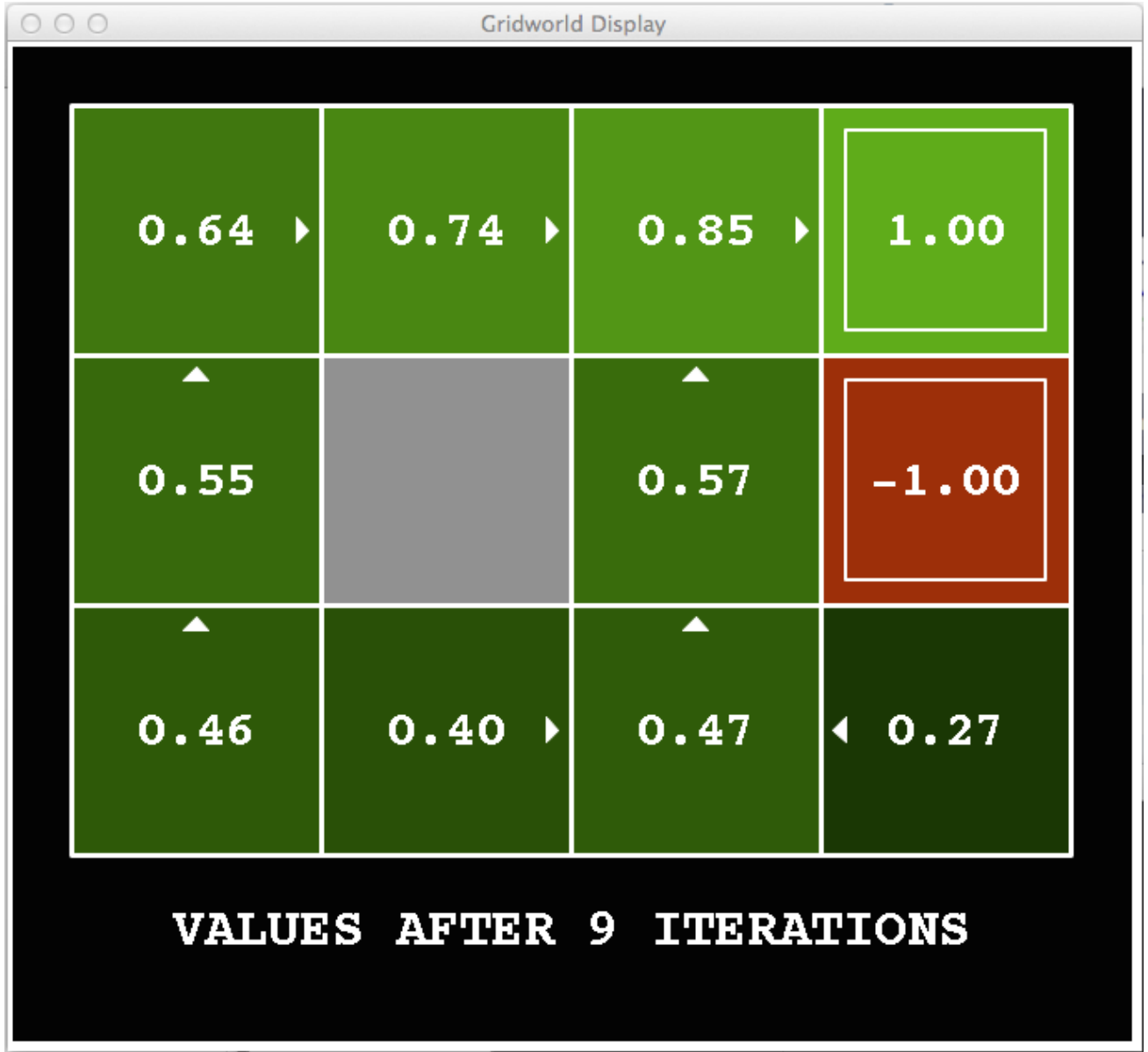

Noise = 0.2
Discount = 0.9
Living reward = 0

# k=8



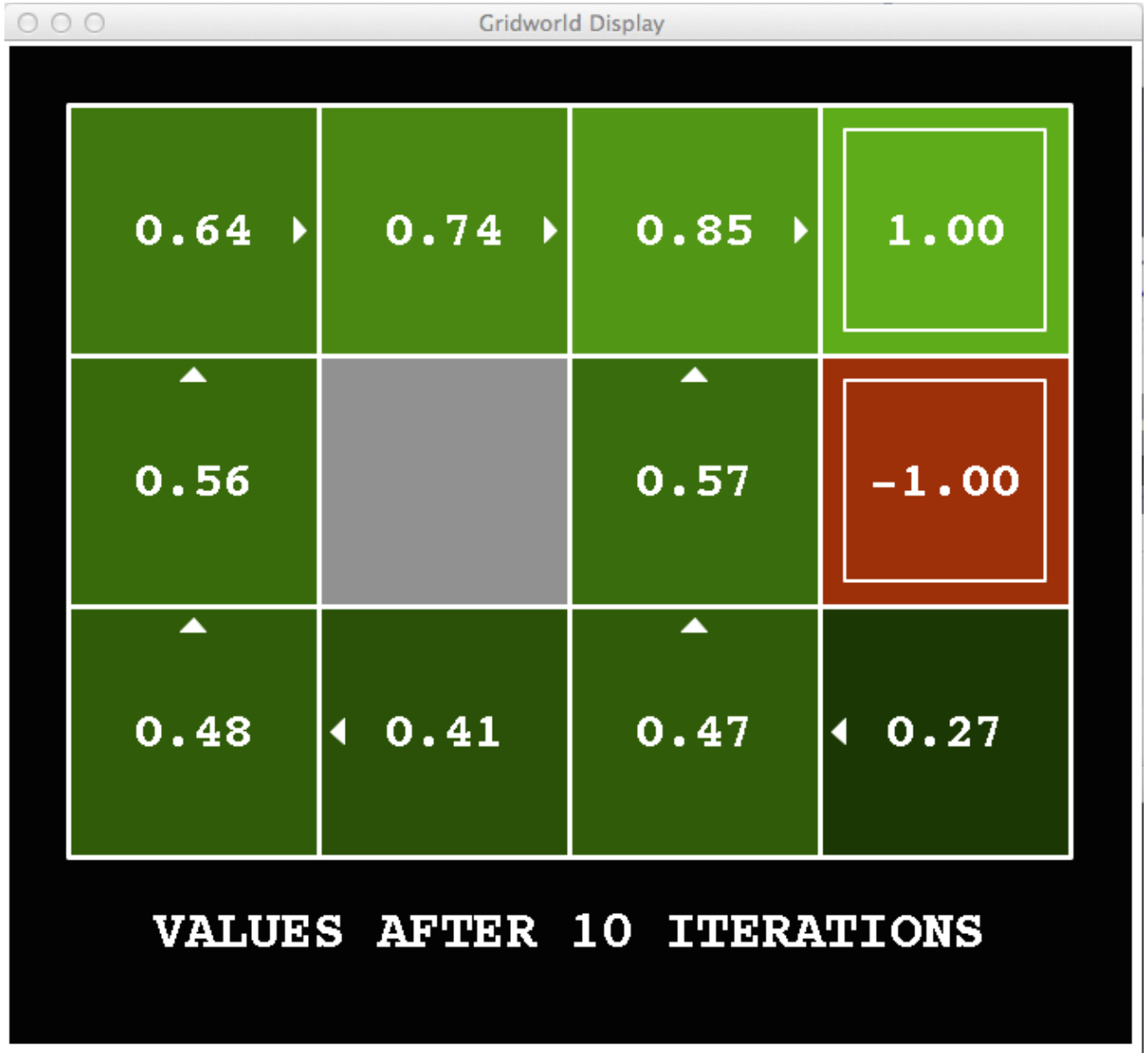

Noise = 0.2
Discount = 0.9
Living reward = 0

# k=9




Noise = 0.2
Discount = 0.9
Living reward = 0

# k=10



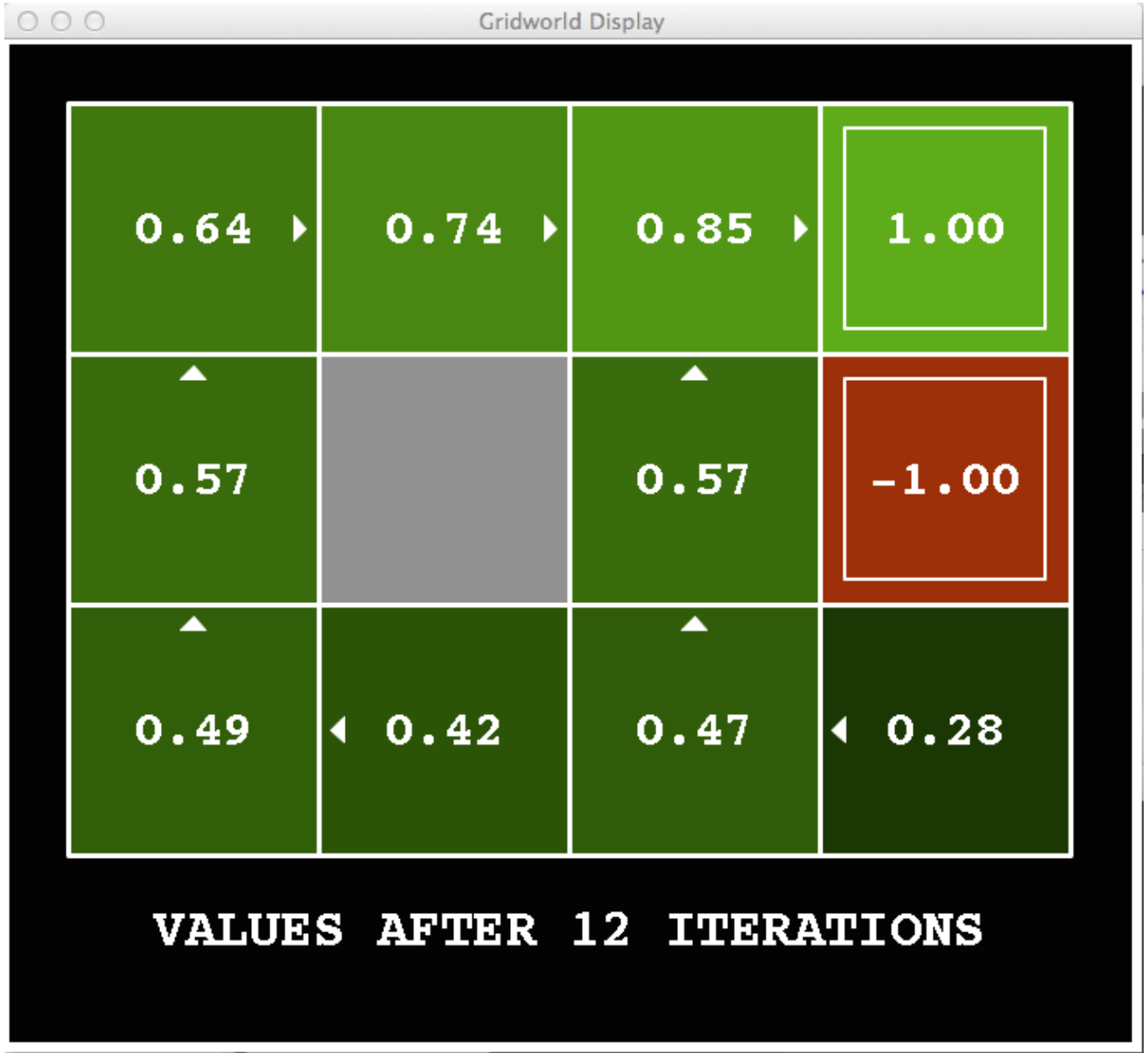

Noise = 0.2
Discount = 0.9
Living reward = 0

# k=11



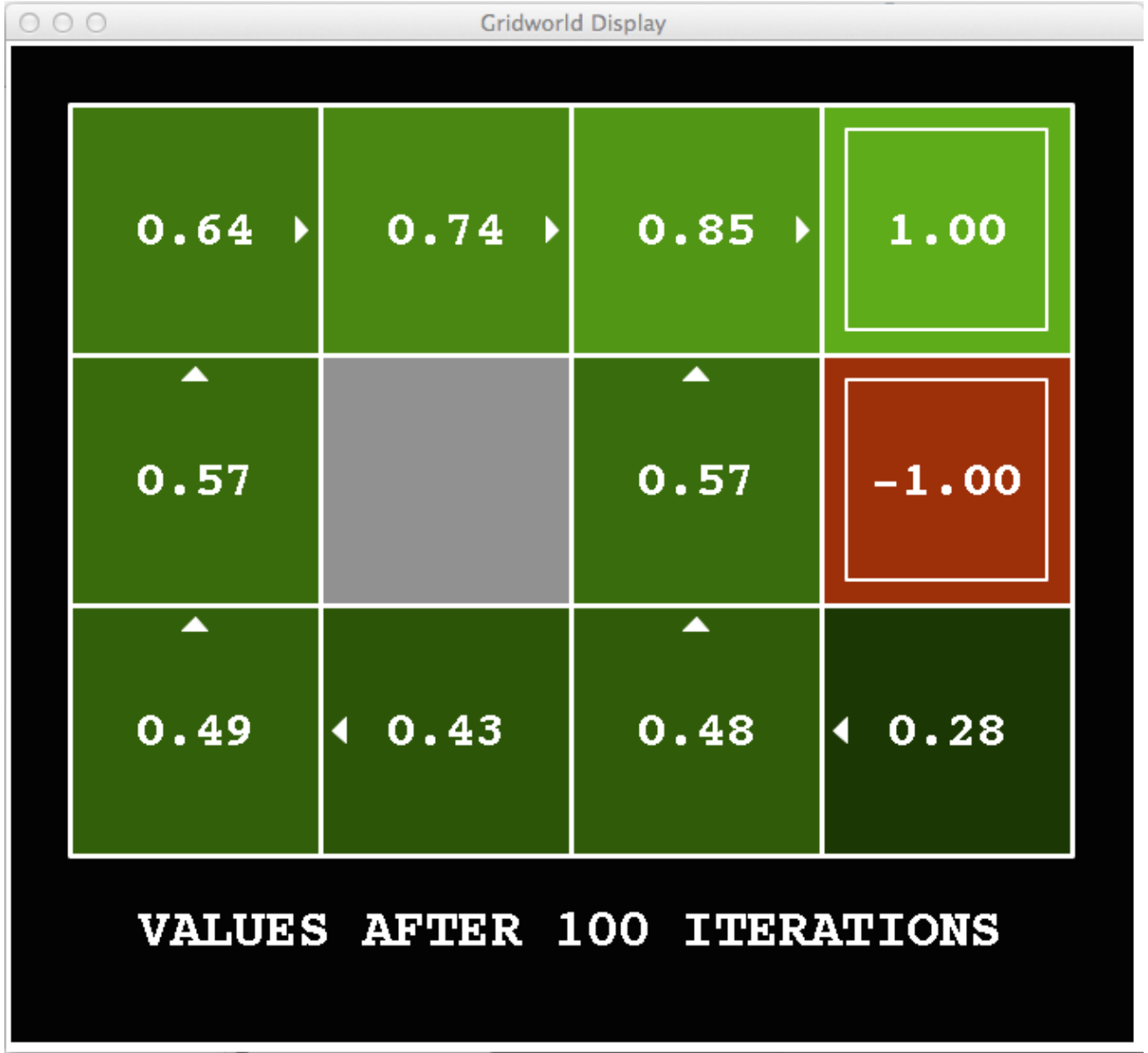

Noise = 0.2
Discount = 0.9
Living reward = 0

# k=12




Noise = 0.2
Discount = 0.9
Living reward = 0

# k=100

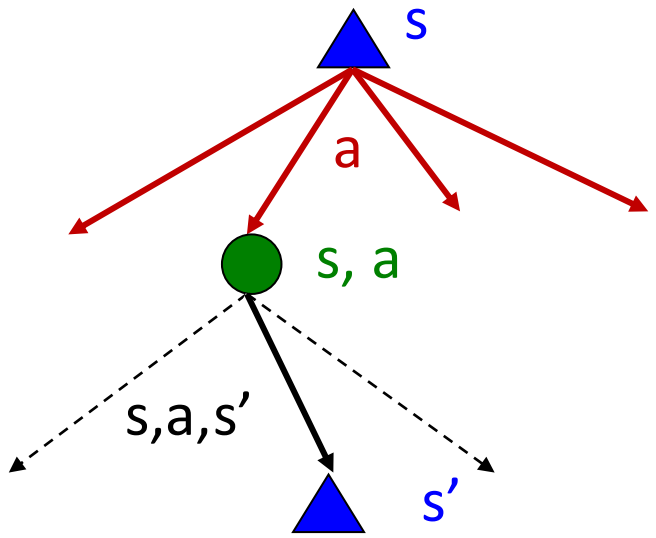



Noise = 0.2
Discount = 0.9
Living reward = 0

# Problems with Value Iteration

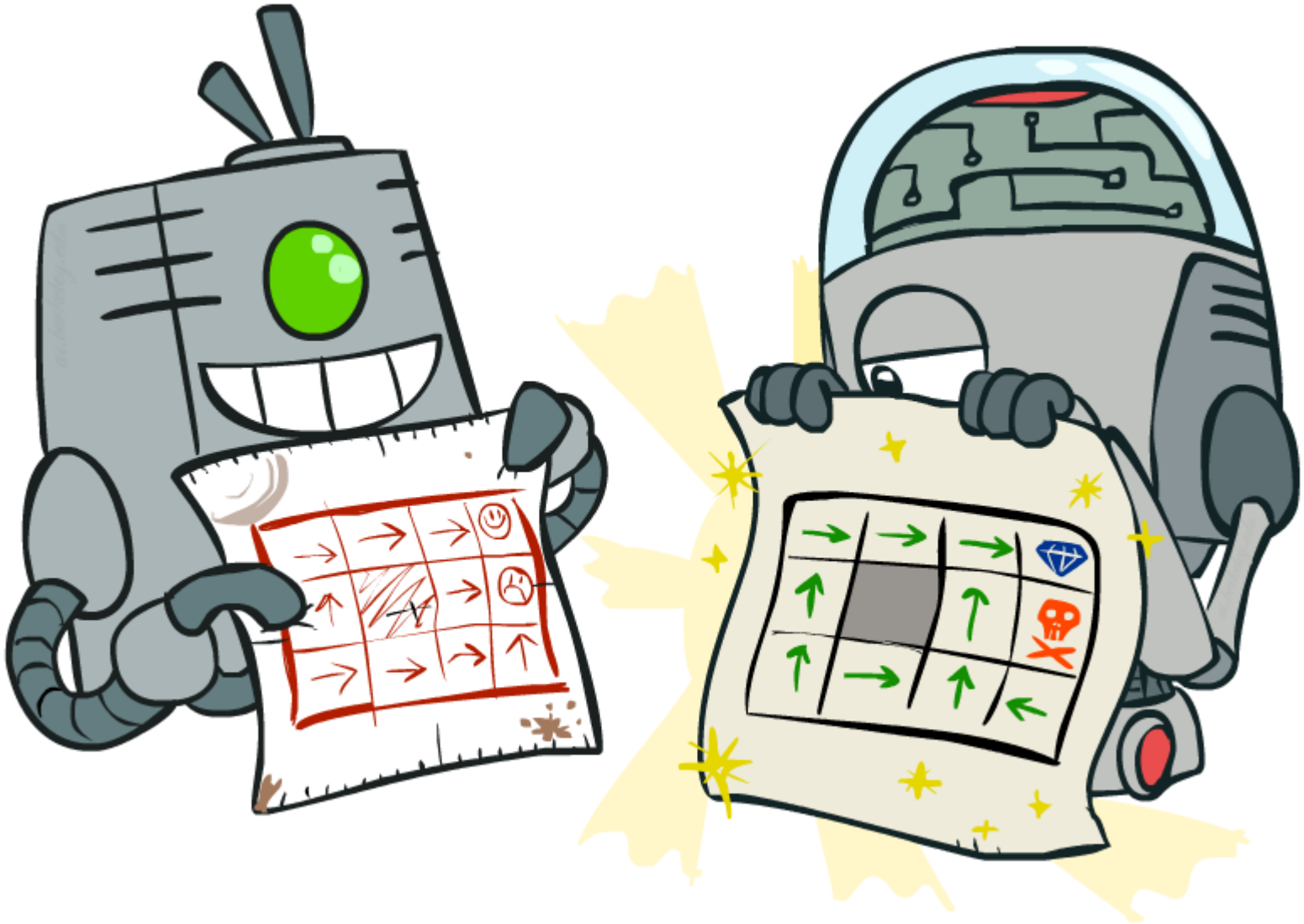
- Value iteration repeats the Bellman updates:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[ R(s,a,s') + \gamma V_k(s') \right]$$

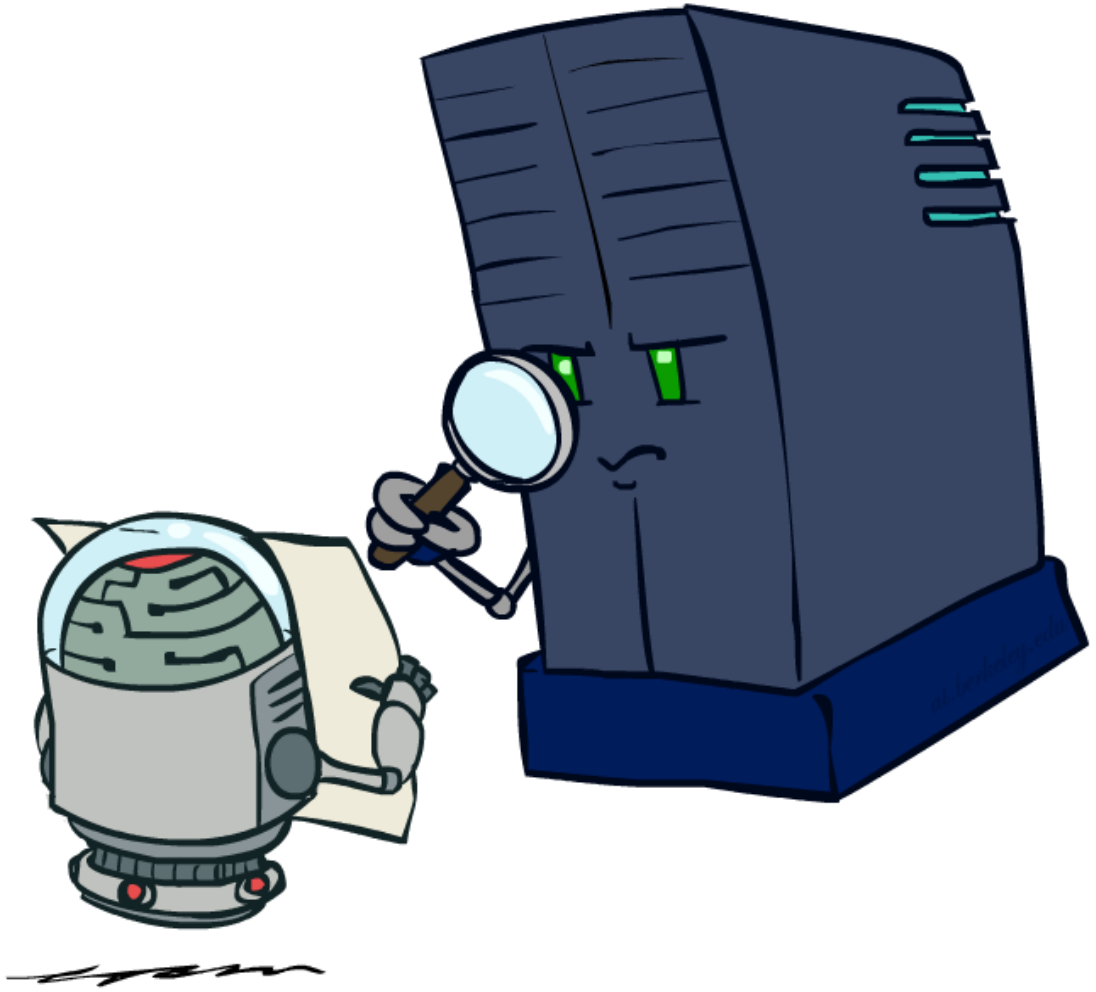



- Problems:
  - It's slow – $O(S^2A)$ per iteration
  - The policy often converges long before the values

# Policy Methods

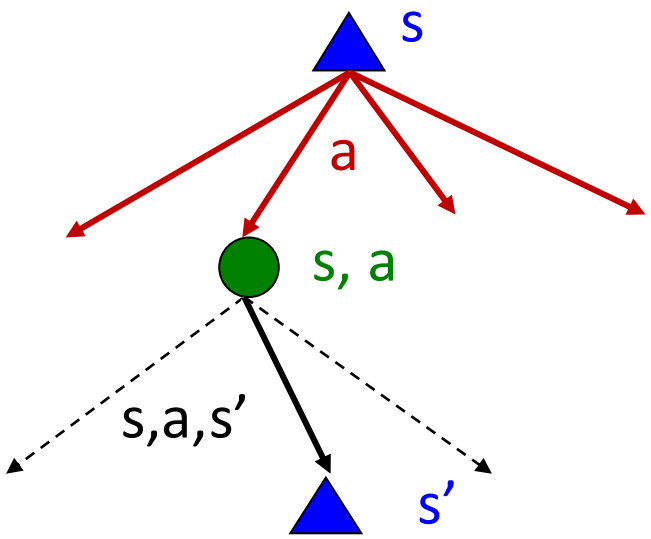


# Policy Evaluation

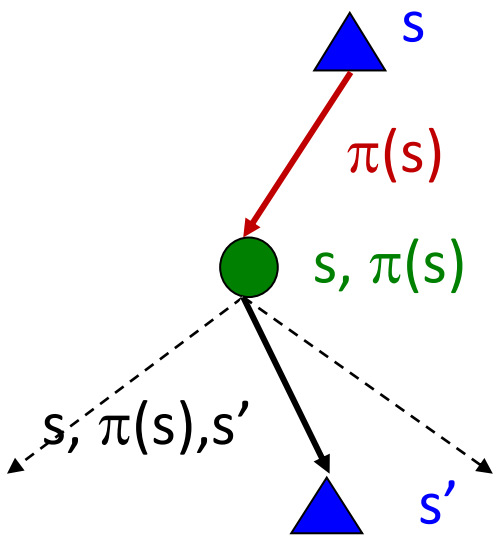


# Fixed Policies

Do the optimal action




Do what $\pi$ says to do




- Expectimax trees max over all actions to compute the optimal values
- If we fixed some policy $\pi(s)$, then the tree would be simpler – only one action per state
  - … though the tree's value would depend on which policy we fixed

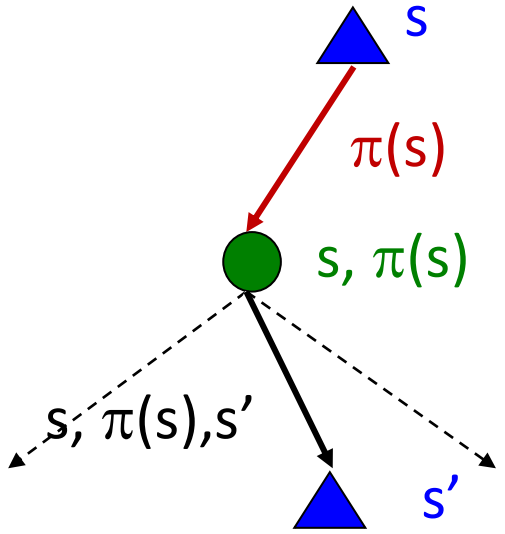
# Utilities for a Fixed Policy

- Another basic operation: compute the utility of a state s under a fixed (generally non-optimal) policy

- Define the utility of a state s, under a fixed policy $\pi$:

  $V^{\pi}(s)$ = expected total discounted rewards starting in s and following $\pi$

- Recursive relation (one-step look-ahead / Bellman equation):

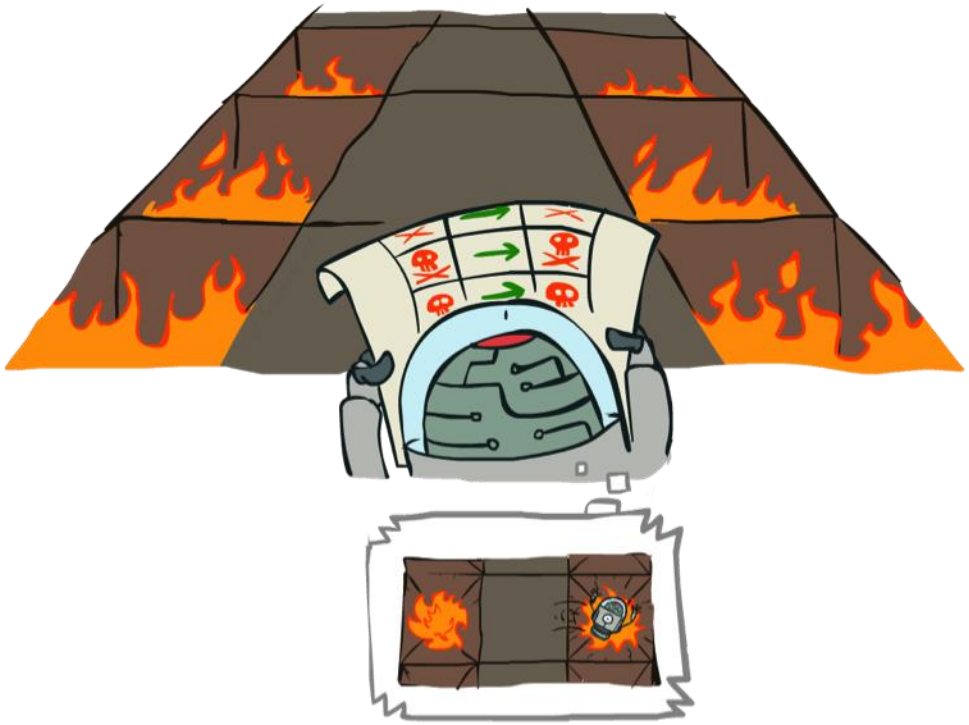



$$V^*(s) = \max_a \sum_{s'} T(s,a,s') \left[ R(s,a,s') + \gamma V^*(s') \right]$$

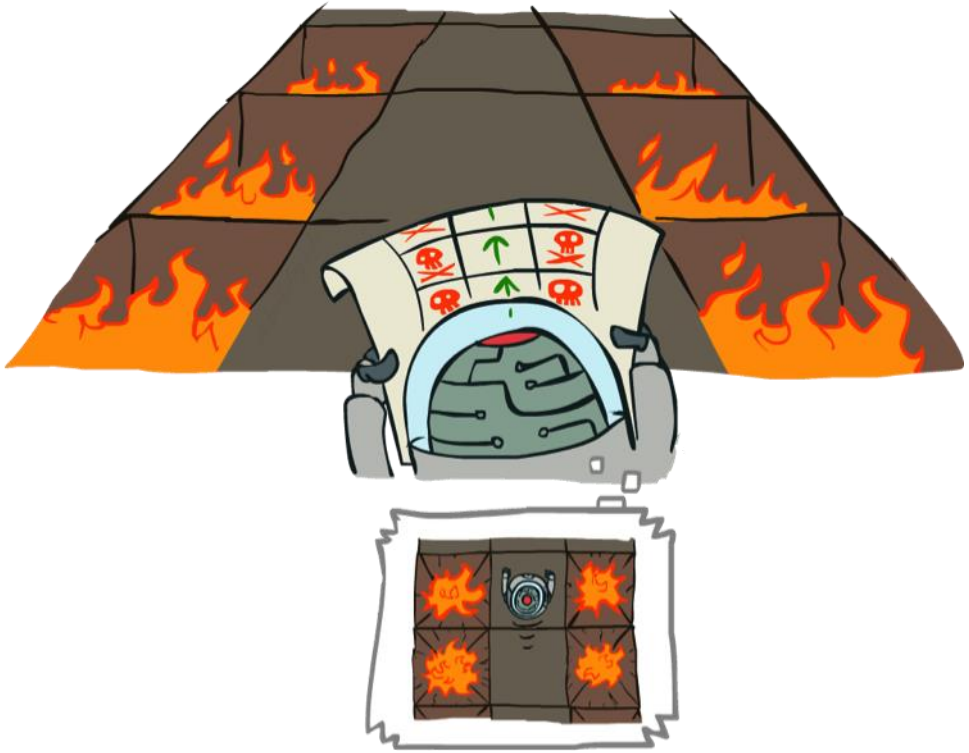
$$V^{\pi}(s) = \sum_{s'} T(s,\pi(s),s')[R(s,\pi(s),s') + \gamma V^{\pi}(s')]$$

# Example: Policy Evaluation

Always Go Right



Always Go Forward



# Example: Policy Evaluation

Always Go Right




Always Go Forward



# Policy Evaluation: A Linear System

**Fixed policy: always go north (↑). Discount γ = 0.9.**

Move reward = 0. Go north with probability 0.8; go left or right with 0.1 each.

| | | |
|---|---|---|
| −10 | 100 | −10 |
| −10 | x ↑ | −10 |
| −10 | y ↑ | −10 |
| −10 | z ↑ | −10 |

Exit-cell values are known: 100 and −10.

**One equation for each unknown value**

$$\begin{cases} x = 0.9[0.8(100) + 0.2(-10)] = 70.20 \\ y = 0.9[0.8x + 0.2(-10)] = 0.72x - 1.8 \\ z = 0.9[0.8y + 0.2(-10)] = 0.72y - 1.8 \end{cases}$$

Rearrange: $y - 0.72x = -1.8$; $z - 0.72y = -1.8$

**Solve: x = 70.20, y = 48.74, z = 33.30**

Fixed policy → no max. Only x, y and z are unknown; their coefficients are known.

These are linear equations, so we can solve them directly—no value iteration needed.

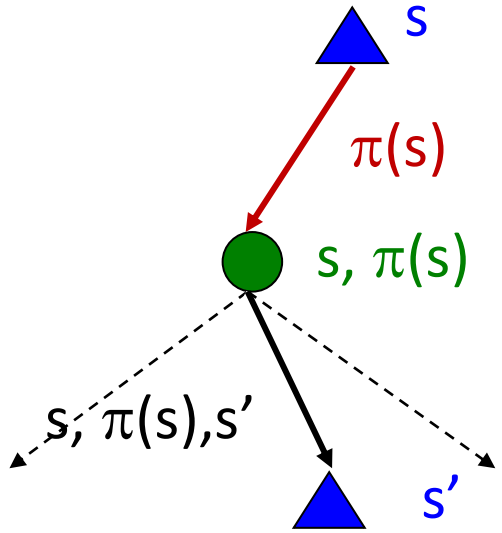
# Policy Evaluation

- How do we calculate the V's for a fixed policy $\pi$?

- Idea 1: Turn recursive Bellman equations into updates (like value iteration)

$$V_0^\pi(s) = 0$$

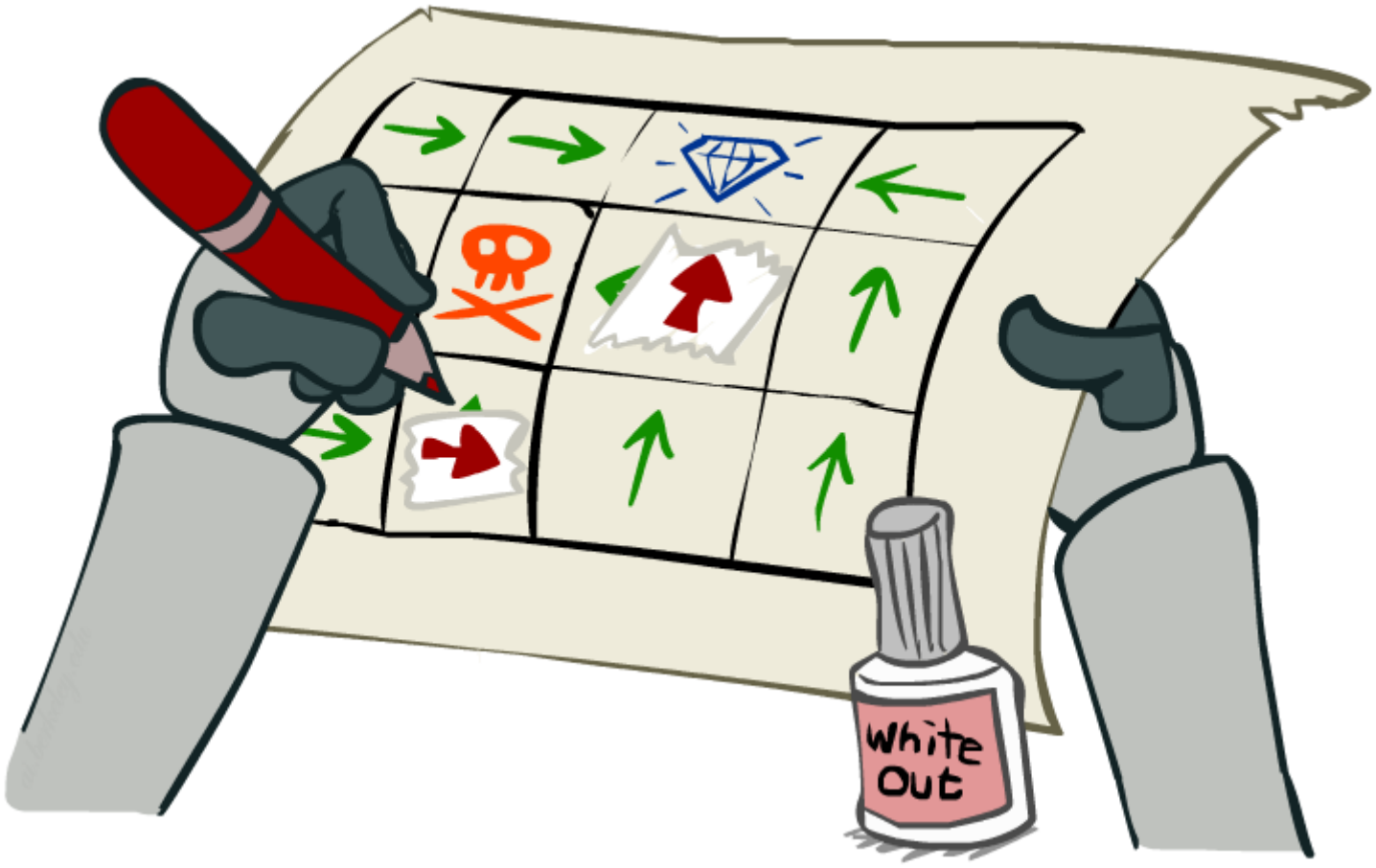
$$V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma V_k^\pi(s')]$$




- Efficiency: O($S^2$) per iteration

- Idea 2: Without the maxes, the Bellman equations are just a linear system

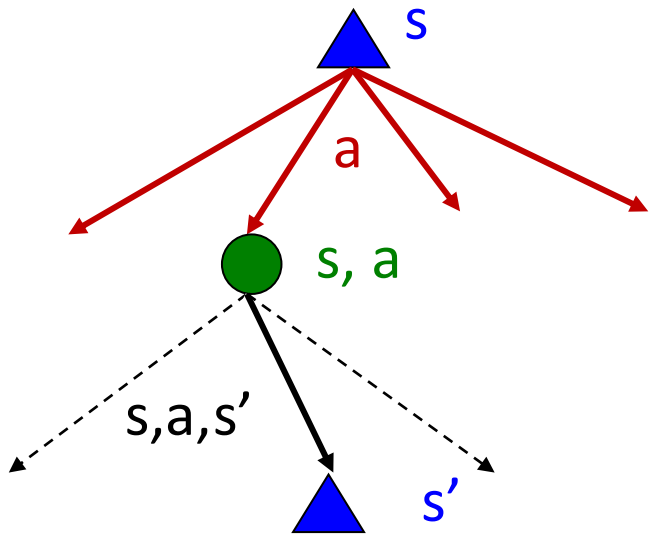
# Policy Iteration



# Problems with Value Iteration

- Value iteration repeats the Bellman updates:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[ R(s,a,s') + \gamma V_k(s') \right]$$




- Problem 1: It's slow – $O(S^2A)$ per iteration
- Problem 2: The "max" at each state rarely changes
- Problem 3: The policy often converges long before the values

# Policy Iteration

- Alternative approach for optimal values:
  - **Step 1: Policy evaluation:** calculate utilities for some fixed policy (not optimal utilities!) until convergence
  - **Step 2: Policy improvement:** update policy using one-step look-ahead with resulting converged (but not optimal!) utilities as future values
  - Repeat steps until policy converges

- This is **policy iteration**
  - It's still optimal!
  - Can converge (much) faster under some conditions

# Policy Iteration

- Evaluation: For fixed current policy $\pi$, find values with policy evaluation:
  - Iterate until values converge:

$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T(s, \pi_i(s), s') \left[ R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s') \right]$$

- Improvement: For fixed values, get a better policy using policy extraction
  - One-step look-ahead:

$$\pi_{i+1}(s) = \arg\max_a \sum_{s'} T(s, a, s') \left[ R(s, a, s') + \gamma V^{\pi_i}(s') \right]$$

Demo: https://nowke.github.io/rlviz

# Comparison

- Both value iteration and policy iteration compute the same thing (all optimal values)
- In value iteration:
  - Every iteration updates both the values and (implicitly) the policy
  - We don't track the policy, but taking the max over actions implicitly recomputes it
- In policy iteration:
  - We do several passes that update utilities with fixed policy (each pass is fast because we consider only one action, not all of them)
  - After the policy is evaluated, a new policy is chosen (slow like a value iteration pass)
  - The new policy will be better (or we're done)
- Both are dynamic programs for solving MDPs

# Summary: MDP Algorithms

- So you want to….
  - Compute optimal values: use value iteration or policy iteration
  - Compute values for a particular policy: use policy evaluation
  - Turn your values into a policy: use policy extraction

- These all look the same!
  - They basically are – they are all variations of Bellman updates
  - They all use one-step lookahead expectimax fragments
  - They differ only in whether we plug in a fixed policy or max over actions

# Next Lecture: Reinforcement Learning

